

MyMquid

NEW FEATURES — API SPECIFICATION

User Management & File Upload

API specification for the two newly built frontend features that require new backend endpoints: the **User Management** system (staff accounts, roles, performance monitoring) and the **File Upload** service (blog featured images, avatars, OG images).

Version 1.0

June 2026

NestJS

JWT Auth

REST

⚠ These endpoints do not yet exist on the backend. This document defines the exact contract the frontend expects — implement accordingly.

Contents

Two feature groups · 11 new endpoints · 1 invite flow

01 — OVERVIEW & DATA MODELS

Base URL & Authentication	3
UserWithStats model	3
CreateUserPayload / UpdateUserPayload	3
Role-Based Access Control	4

02 — USER MANAGEMENT API NEW

GET /users — List all users with stats	5
GET /users/:id — Get user with stats	5
POST /users — Create user + send invite	6
PUT /users/:id — Update user	6
PATCH /users/:id/status — Activate / Deactivate	7
DELETE /users/:id — Delete user	7
POST /users/:id/reset-password — Trigger reset email	7
GET /users/:id/posts — User's blog posts	8

03 — INVITE / SET PASSWORD FLOW NEW

POST /auth/set-password — First-time password setup	8
End-to-end invite flow diagram	9

04 — FILE UPLOAD API NEW

POST /upload — Upload image file	10
Upload types & storage notes	10

05 — QUICK REFERENCE

All new endpoints at a glance	11
-------------------------------	----



01 — Overview & Data Models

Base URL, authentication requirements, and TypeScript contracts

Base URL & Authentication

BASE URL

```
https://mquid.onrender.com/api/v1
```

ALL REQUESTS INCLUDE

```
Content-Type: application/json
Authorization: Bearer <jwt_token>
```

TOKEN SOURCE

The frontend reads the JWT from `localStorage` key `mymquid-admin-auth` (Zustand persist state). It is injected into every request automatically via the Axios interceptor in `src/lib/axios.ts`.

All `/users` endpoints require role: `"super_admin"` in the JWT payload. Return 403 Forbidden for any other role.

UserWithStats

Response shape for all `/users` endpoints

TYPESCRIPT DEFINITION

```
type UserWithStats = {
  id: string; // UUID
  name: string;
  email: string;
  role: "super_admin" | "staff";
  avatar?: string; // URL or
  null
  active: boolean; // account
  status
  lastLogin: string; // ISO 8601
  createdAt: string; // ISO 8601
  stats: {
    published: number;
    drafts: number;
    scheduled: number;
    total: number;
  };
};
```

EXAMPLE JSON

```
{
  "id": "a1b2c3",
  "name": "Jane Staff",
  "email": "jane@mymquid.com",
  "role": "staff",
  "avatar": null,
  "active": true,
  "lastLogin": "2026-05-18T14:00:00Z",
  "createdAt": "2026-02-15T00:00:00Z",
  "stats": {
    "published": 3,
    "drafts": 2,
    "scheduled": 0,
    "total": 5
  }
}
```

stats must be pre-computed by the backend from the `blog_posts` table. Count posts `WHERE author_id = :id` grouped by status. `total` = sum of all statuses. Do NOT make the frontend compute these from a list of posts.

CreateUserPayload & UpdateUserPayload

Request bodies for POST and PUT /users

CREATEUSERPAYLOAD (POST /USERS)

```
{
  "name": "Jane Staff",    // required,
  min 2
  "email": "jane@mymquid.com", //
  required, unique
  "role": "staff"          // required
}
```

No password field — account is created with no password. Backend sends a "Set your password" invite email. See Section 03 for the full flow.

UPDATEUSERPAYLOAD (PUT /USERS/:ID)

```
{
  "name": "Jane Updated", // required
  "email": "jane@mymquid.com", // required
  "role": "super_admin"    // required
}
```

The frontend disables the email field in edit mode, but the email is still included in the payload. If the backend allows email changes, validate uniqueness.

Role-Based Access Control for User Management

ENDPOINT GROUP	SUPER_ADMIN	STAFF	UNAUTHENTICATED
GET /users	✓ Allowed	⊘ 403	⊘ 401
GET /users/:id	✓ Allowed	⊘ 403	⊘ 401
POST /users	✓ Allowed	⊘ 403	⊘ 401
PUT /users/:id	✓ Allowed	⊘ 403	⊘ 401
PATCH /users/:id/status	✓ Allowed	⊘ 403	⊘ 401
DELETE /users/:id	✓ Allowed	⊘ 403	⊘ 401
POST /users/:id/reset-password	✓ Allowed	⊘ 403	⊘ 401
GET /users/:id/posts	✓ Allowed	⊘ 403	⊘ 401
POST /auth/set-password	✓	✓	✓ (invite token)
POST /upload	✓ Allowed	✓ Allowed	⊘ 401

The frontend enforces this via RoleGuard in the UI, but the backend MUST also enforce role checks independently. Never rely solely on frontend guards for security.



02 — User Management API NEW

Full lifecycle management for staff and admin accounts — super_admin only

GET /users

super_admin only List all users with stats

QUERY PARAMETERS

PARAM	TYPE	DESCRIPTION
page	number	Page number, default 1
limit	number	Items per page, default 20

RESPONSE 200 OK

```
[
  /* UserWithStats[] — see model in §01
  */
]
```

Returns all users (no status filter). Sorted by createdAt DESC. The frontend displays all users regardless of active status.

GET /users/:id

super_admin only Single user with pre-computed stats

PATH PARAMETER

:id	User UUID
-----	-----------

RESPONSES

200	Full <code>UserWithStats</code> object
404	User not found

This endpoint is called when the super admin clicks a user row to open the detail page (/admin/users/:id). The stats object must be populated — it drives the 4 stat cards (Published, Drafts, Scheduled, Last Login) at the top of the page.

POST /users

super_admin only Create account + send invite email

REQUEST BODY

```
{
  "name": "Jane Staff",
  "email": "jane@mymquid.com",
  "role": "staff"
}
```

RESPONSES

201	Created <code>UserWithStats</code> object
409	{ "message": "Email already in use" }
400	Validation error

SERVER RESPONSIBILITIES

- Generate UUID for `id`, set `createdAt` to `now()`
- Set `active: true`, `lastLogin: null` (or current time), stats all zeros
- Create account with no password — do NOT require a password in this request
- Generate a time-limited invite token (e.g. JWT or UUID, expires in 24–48h)
- Send email to `email` with a link: `https://[domain]/admin/set-password?token=[invite_token]`
- Return the created `UserWithStats` object — frontend shows success toast immediately

Frontend toast on success: *"User created. An invite email has been sent to jane@mymquid.com."*

PUT

`/users/:id`

super_admin only

Update name, email, or role

REQUEST BODY

```
{
  "name": "Jane Updated",
  "email": "jane@mymquid.com",
  "role": "super_admin"
}
```

RESPONSES

- | | |
|-----|---|
| 200 | Updated <code>UserWithStats</code> object |
| 404 | User not found |
| 409 | Email taken by another user |



02 — User Management API (continued)

Status toggle, delete, password reset, and user posts

PATCH

/users/:id/status

super_admin only

Activate or deactivate a user account

REQUEST BODY

```
{
  "active": false // true to activate,
                false to deactivate
}
```

RESPONSES

200	Updated <code>UserWithStats</code> object with new <code>active</code> value
404	User not found

DEACTIVATION BEHAVIOUR

- Set `active = false` on the user record
- Invalidate any active JWT sessions for that user
- The deactivated user can no longer log in (401 on next login attempt)
- Their existing posts are NOT deleted

Self-deactivation guard: Prevent a `super_admin` from deactivating their own account — return 400 { "message": "Cannot deactivate your own account" }.

DELETE

/users/:id

super_admin only

Permanently delete a user

PATH PARAMETER

<code>:id</code>	User UUID
------------------	-----------

RESPONSES

204	Deleted — no body
404	User not found
400	Cannot delete own account

SERVER RESPONSIBILITIES

- Delete user record from `users` table
- Decide on cascade: reassign posts to a system user, or delete them
- Invalidate all active sessions for the user

On success: The frontend redirects the `super_admin` to `/admin/users` and removes the user from the local list optimistically.

POST

/users/:id/reset-password

super_admin only

Trigger password reset email for any user

REQUEST

No body required. `:id` identifies the target user.

RESPONSES

200	{ "message": "Reset email sent" }
-----	-----------------------------------

SERVER RESPONSIBILITIES

- Generate a time-limited reset token (24h expiry)
- Send email to the user's registered address
- Link points to: [domain]/admin/reset-password?token=[token]

404

⚠ These endpoints are not yet implemented on the backend
User not found

Frontend toast on success: "Password reset email sent." This is different from POST /auth/forgot-password — here the super admin triggers it on behalf of another user (not self-service).

GET

/users/:id/posts

super_admin only

All blog posts authored by this user

QUERY PARAMETERS

page

default 1

limit

default 20

RESPONSE 200 OK

Array of BlogPost objects (same shape as GET /blog items).

USAGE ON THE FRONTEND

Called when the User Detail page loads. Results populate two UI areas:

Overview tab — "Recent Activity" card shows the last 5 posts (sorted by updatedAt DESC).

Blog Posts tab — Full table of all posts with Edit and View links.



03 — Invite / Set Password Flow NEW

How a newly created user sets their password for the first time

POST **/auth/set-password** No auth — uses one-time invite token

REQUEST BODY

```
{
  "token":
  "invite_token_from_email",
  "password":
  "new_secure_password",
  "confirmPassword":
  "new_secure_password"
}
```

RESPONSES

- 200** Password set — return { user, token } (auto-login)
- 400** Passwords don't match or too weak
- 400** Token expired or already used
- 404** Token not found

VALIDATION RULES

token	Valid, non-expired invite token
password	Min 8 chars, must include number or symbol
confirmPassword	Must match password

Recommended: On success, return { user, token } so the frontend can auto-login the user and redirect them to /admin/dashboard without requiring a separate login step.

End-to-End Invite Flow

- 1 Super admin opens "Add User" modal**
Frontend: /admin/users → clicks "Add User" → fills name, email, role → submits
- 2 Frontend calls POST /users**
Body: { name, email, role } — no password
- 3 Backend creates user record + generates invite token**
Stores token with user_id, expires_at = now() + 48h, used = false
- 4 Backend sends invite email to the new user**
Link: https://[domain]/admin/set-password?token=[token]

5 Backend returns 201 with the created UserWithStats

Frontend: modal closes, user appears in the list, toast "Invite email sent to jane@mymquid.com"

**6 New user clicks the link in their email**

Browser opens: `/admin/set-password?token=[token]` — frontend shows a "Set Password" form

**7 New user submits password form**

Frontend calls: `POST /auth/set-password` with `{ token, password, confirmPassword }`

**8 Backend validates token, sets password, marks token as used**

Hash password with `bcrypt`, update user record, invalidate invite token

**9 Backend returns 200 with { user, token } — user is auto-logged in**

Frontend stores auth in Zustand and redirects to `/admin/dashboard`

Token security: Use a cryptographically random token (UUID v4 or similar), not a predictable value. Mark it as used after first successful `POST /auth/set-password` call. Expire after 48 hours. Return a generic error for invalid/expired tokens — do not reveal whether the token exists.

Frontend Page Required: Set Password

A `/admin/set-password` page needs to be added to the frontend to handle the invite link. It reads the `?token=` query param, shows a two-field form (password + confirm password), and calls `POST /auth/set-password`. This page is NOT yet implemented on the frontend — it should be built alongside the backend endpoint.



04 — File Upload API NEW

Image uploads for blog featured images, OG images and user avatars

POST

/upload

Auth Required

Upload an image file, return public URL

REQUEST — MULTIPART/FORM-DATA

FIELD	TYPE	REQUIRED	DESCRIPTION
file	File (binary)	Required	Image file. Accepted: JPEG, PNG, WebP, GIF. Max size: 5 MB.
type	string enum	Required	One of: "blog-image" , "avatar" , "og-image"

EXAMPLE REQUEST (AXIOS)

```
const form = new FormData();
form.append("file", file);           // File object from input[type=file]
form.append("type", "blog-image");

await axios.post("/upload", form, {
  headers: { "Content-Type": "multipart/form-data" },
});
```

RESPONSE 201 CREATED

```
{
  "url":
  "https://cdn.mymquid.com/blog/abc123.jpg"
}
```

The frontend stores this URL in the form's featuredImage field and displays a thumbnail. The URL is sent as a string in the blog post payload.

ERROR RESPONSES

400	File missing, invalid type, or unsupported format
413	File too large (over 5 MB)
401	Not authenticated

Upload Types & Usage

TYPE VALUE	USED BY	WHERE URL IS STORED	RECOMMENDED SIZE
"blog-image"	Blog Editor sidebar "Featured Image" card	BlogPost.featuredImage field (string URL)	1200×630 px (16:9)
"og-image"	Blog Editor SEO section OG Image field	BlogPost.seo.ogImage field	1200×630 px

TYPE VALUE	USED BY	WHERE URL IS STORED	RECOMMENDED SIZE
"avatar"	Profile page avatar upload (future)	<code>AdminUser.avatar</code> field	400×400 px (1:1)

Storage Implementation Notes

RECOMMENDED: CLOUD STORAGE

- **AWS S3** — `@aws-sdk/client-s3` + presigned URLs or public bucket
- **Cloudflare R2** — S3-compatible, free egress
- **Supabase Storage** — simple if already using Supabase
- **Cloudinary** — built-in transforms + CDN

MINIMUM REQUIREMENTS

- Upload to storage, return a permanent public HTTPS URL
- Accept: `image/jpeg`, `image/png`, `image/webp`
- Reject non-image MIME types with 400
- Enforce 5 MB max size
- Generate a unique filename (UUID + original extension)

Do NOT store images locally on the NestJS server filesystem — they will be lost on Render.com redeploys. Use a cloud storage provider. The returned URL must be publicly accessible (no auth required to view) since it is rendered on the public blog pages.



05 — Quick Reference

All new endpoints at a glance + frontend integration points

All New Endpoints

METHOD	ENDPOINT	AUTH	ROLE	DESCRIPTION
GET	/users	✓ JWT	super_admin	List all users with stats
GET	/users/:id	✓ JWT	super_admin	Single user with pre-computed stats
POST	/users	✓ JWT	super_admin	Create user + trigger invite email
PUT	/users/:id	✓ JWT	super_admin	Update name / email / role
PATCH	/users/:id/status	✓ JWT	super_admin	Activate or deactivate account
DELETE	/users/:id	✓ JWT	super_admin	Permanently delete user
POST	/users/:id/reset-password	✓ JWT	super_admin	Trigger password reset email for user
GET	/users/:id/posts	✓ JWT	super_admin	Blog posts authored by this user
POST	/auth/set-password	None	Invite token	First-time password setup from invite email
POST	/upload	✓ JWT	Any role	Upload image, receive public URL

Frontend Integration Points

Where each endpoint is called in the codebase

ENDPOINT	CALLED FROM (FRONTEND FILE)	TRIGGERED BY
GET /users	src/admin/users/useUserStore.ts → fetchUsers()	Manage Users page mount

ENDPOINT	CALLED FROM (FRONTEND FILE)	TRIGGERED BY
GET /users/:id	useUserStore.ts → fetchUser(id)	User detail page mount
POST /users	useUserStore.ts → createUser(payload)	"Add User" modal submit
PUT /users/:id	useUserStore.ts → updateUser(id, payload)	"Edit" modal submit
PATCH /users/:id/status	useUserStore.ts → toggleUserStatus(id, active)	Activate/Deactivate confirm
DELETE /users/:id	useUserStore.ts → deleteUser(id)	Delete confirm modal
POST /users/:id/reset-password	useUserStore.ts → resetUserPassword(id)	"Reset Password" confirm
GET /users/:id/posts	useUserStore.ts → fetchUserPosts(id)	User detail page mount
POST /auth/set-password	New page: src/admin/auth/SetPasswordPage.tsx (to be built)	Invite email link
POST /upload	src/admin/mock/api.ts → uploadApi.upload(file, type)	Featured image file picker in blog editor

Implementation Checklist

USERS MODULE

- ☐ Create `users` table / entity
- ☐ Seed initial `super_admin` account
- ☐ `GET /users` with stats aggregation
- ☐ `GET /users/:id` with stats
- ☐ `POST /users` + invite email
- ☐ `PUT /users/:id`
- ☐ `PATCH /users/:id/status`
- ☐ `DELETE /users/:id`
- ☐ `POST /users/:id/reset-password`
- ☐ `GET /users/:id/posts`
- ☐ Role guard: `super_admin` only for all `/users`

AUTH + UPLOAD MODULE

- ☐ `POST /auth/set-password`
- ☐ Invite token table / expiry logic
- ☐ Email service (invite + reset)
- ☐ Frontend: `SetPasswordPage.tsx` (new page)
- ☐ `POST /upload` endpoint
- ☐ Cloud storage integration (S3/R2/Cloudinary)
- ☐ File validation (MIME type + size)
- ☐ Return permanent public URL